Projet de Module : Systèmes Répartis et Distribués

Année: 2026

Master d'Excellence : IL, ADIA & IISE

1. Contexte et objectifs du projet

L'objectif de ce projet est de concevoir, développer, déployer et évaluer une application reposant sur une architecture distribuée moderne. Les étudiants devront mettre en œuvre plusieurs composants logiciels indépendants qui communiquent à travers un réseau afin de fournir un service cohérent.

Les projets devront mobiliser des technologies modernes telles que les microservices, les API REST, gRPC, les systèmes de messagerie asynchrone, la conteneurisation et, selon le sujet choisi, les grands modèles de langage (LLM) ou l'apprentissage distribué.

Ce projet vise à mettre en pratique les concepts fondamentaux des systèmes distribués : communication interservices, répartition des traitements, passage à l'échelle, tolérance aux pannes, gestion des données, supervision et sécurité.

Il permettra également d'explorer les interactions entre l'intelligence artificielle et les systèmes distribués, notamment l'intégration des LLM dans les applications distribuées, l'orchestration de services par des agents intelligents et l'entraînement distribué de modèles d'apprentissage automatique.

Le domaine agricole est proposé comme domaine d'application pour illustrer les différentes catégories de projets. Il n'est pas obligatoire : chaque équipe peut choisir une application dans un autre domaine, sous réserve de validation du sujet par l'enseignant.

2. Organisation du travail

- **Taille des équipes :** 3 à 4 étudiants par projet.
- **Composition des équipes :** tous les membres doivent appartenir au même groupe de TD/TP.
- **Responsable du groupe :** chaque équipe doit désigner un responsable chargé de coordonner le travail et de soumettre le lien du dépôt GitHub sur Classroom.
- **Choix du sujet :** chaque équipe choisit un sujet parmi les propositions ci-dessous ou soumet une proposition équivalente à la validation de l'enseignant.
- **Répartition des responsabilités :** chaque étudiant doit prendre en charge une partie identifiable du projet.

- **Gestion du code :** l'utilisation de GitHub est obligatoire pour le versionnement, la collaboration et le suivi des contributions.

Les responsabilités pourront notamment couvrir la conception de l'architecture, le développement des services, l'intégration de l'IA, le déploiement, les tests et la supervision. La répartition doit rester adaptée à la taille de l'équipe et ne doit pas empêcher chaque étudiant de comprendre l'architecture globale.

3. Sujets de projets proposés

Les projets sont répartis en quatre catégories. Pour chaque catégorie, des exemples d'application agricole sont proposés à titre indicatif. Les équipes peuvent choisir un autre domaine d'application ou proposer un sujet équivalent.

A. Applications distribuées fondées sur les microservices

Projet 1 — Plateforme distribuée de réservation

Développer une application de réservation de salles, de ressources universitaires ou d'événements, organisée en plusieurs microservices indépendants : utilisateurs, disponibilités, réservations et notifications.

Exemple dans le domaine agricole : développer une plateforme de réservation de matériel agricole partagé, permettant aux exploitants de consulter les disponibilités des tracteurs, des machines et des équipements d'irrigation, puis de réserver ces ressources. Des microservices distincts gèrent les utilisateurs, le matériel, les réservations et les notifications.

Projet 2 — Système distribué de gestion de commandes

Développer une application de commerce électronique composée de services indépendants pour les produits, les commandes, les paiements simulés et la gestion des stocks. Le projet devra traiter les problèmes de communication interservices et de cohérence des données.

Exemple dans le domaine agricole : développer une plateforme de commercialisation de produits agricoles reliant producteurs, coopératives et acheteurs. Des microservices gèrent les produits disponibles, les commandes, les stocks et le suivi des livraisons.

Projet 3 — Plateforme de supervision de données IoT

Développer une application capable de recevoir des données provenant de plusieurs capteurs simulés, de les traiter et de les stocker dans des services indépendants. Une interface permettra de visualiser les données et de suivre l'état du système.

Exemple dans le domaine agricole : concevoir une plateforme de surveillance de plusieurs parcelles agricoles à partir de capteurs simulés mesurant l'humidité du sol, la température et l'humidité de l'air. Le système collecte les mesures, les stocke et déclenche des alertes lorsque certaines valeurs dépassent des seuils définis.

B. Intégration des LLM dans les systèmes distribués

Projet 4 — Assistant intelligent fondé sur une architecture de microservices

Développer un assistant utilisant un LLM pour répondre aux demandes des utilisateurs et solliciter plusieurs services spécialisés, par exemple un service de recherche documentaire, un service de gestion des utilisateurs et un service de consultation de données.

Exemple dans le domaine agricole : développer un assistant agricole capable de répondre aux questions des exploitants en consultant plusieurs services : données météorologiques, informations sur les parcelles, mesures des capteurs et documentation agronomique. Le LLM formule une réponse à partir des informations récupérées par les services.

Projet 5 — Système distribué de recherche documentaire avec génération augmentée par récupération (RAG)

Développer une plateforme dans laquelle les documents sont indexés et recherchés par des composants indépendants. Un service utilisant un LLM génère des réponses à partir des documents retrouvés. Le projet devra distinguer la recherche, la récupération du contexte et la génération de réponses.

Exemple dans le domaine agricole : développer un assistant documentaire qui répond aux questions sur les pratiques culturales, l'irrigation, la fertilisation et la protection des cultures à partir de documents techniques. Des composants distincts assurent l'indexation, la recherche des passages pertinents et la génération de réponses par le LLM.

Projet 6 — Orchestration intelligente de services par un LLM

Développer une application dans laquelle un LLM analyse une demande en langage naturel et sélectionne les services nécessaires pour exécuter une tâche. Les services pourront, par exemple, rechercher des informations, effectuer des calculs ou consulter une base de données. Les actions devront être contrôlées et les erreurs de communication prises en compte.

Exemple dans le domaine agricole : un utilisateur demande à l'assistant d'évaluer les besoins en irrigation d'une parcelle. Le LLM organise les appels aux services de consultation des mesures du sol, de récupération des données météorologiques et de calcul d'une recommandation. Le système doit gérer l'indisponibilité éventuelle d'un service et contrôler les actions proposées.

C. LLM et calcul distribué

Projet 7 — Plateforme de génération de texte avec service LLM distant

Développer une application dont l'interface utilisateur est séparée du service d'inférence LLM. Le système devra gérer plusieurs requêtes simultanées, mesurer les temps de réponse et étudier les effets de la charge sur le service. Une solution locale, par exemple avec Ollama, ou un service accessible par API pourra être utilisé.

Exemple dans le domaine agricole : développer une application permettant à plusieurs utilisateurs de demander simultanément des synthèses de rapports agricoles ou des explications sur les mesures de leurs parcelles. L'interface, le service de gestion des requêtes et le serveur LLM sont séparés afin d'évaluer les délais de réponse, le débit et le comportement sous forte charge.

Projet 8 — Étude expérimentale de l'inférence distribuée des LLM

Concevoir un prototype illustrant la répartition du traitement d'un modèle de langage entre plusieurs composants ou processus. Selon les ressources disponibles, l'étude pourra porter sur la répartition des requêtes entre plusieurs instances du modèle, le parallélisme de pipeline ou la répartition des traitements. Une comparaison expérimentale devra préciser les gains et les limites de la solution retenue.

Exemple dans le domaine agricole : concevoir un service qui traite les demandes de plusieurs exploitations agricoles en répartissant les requêtes entre plusieurs instances d'un LLM. L'équipe comparera différentes stratégies de répartition et mesurera les temps de réponse, le débit et l'utilisation des ressources.

L'objectif est d'étudier le calcul distribué et les stratégies de service. Il n'est pas nécessaire d'entraîner un grand modèle de langage ni de disposer d'un cluster GPU.

D. Apprentissage distribué

Projet 9 — Apprentissage fédéré sur plusieurs clients

Développer un système d'apprentissage fédéré dans lequel plusieurs clients entraînent localement un modèle à partir de leurs propres données. Un serveur central agrège les paramètres des modèles locaux, par exemple avec l'algorithme FedAvg. Le projet devra étudier l'évolution des performances en fonction du nombre de clients et de la répartition des données.

Exemple dans le domaine agricole : simuler plusieurs exploitations disposant chacune de données locales sur les conditions de culture. Chaque client entraîne localement un modèle de prédiction, puis transmet ses mises à jour à un serveur d'agrégation. Les données brutes restent locales pendant le processus d'apprentissage.

Projet 10 — Apprentissage distribué pour la détection d'anomalies

Développer un système dans lequel plusieurs clients ou nœuds participent à l'entraînement d'un modèle de détection d'anomalies. Les étudiants devront mettre en place une stratégie d'entraînement et d'agrégation, puis évaluer les performances du modèle et les coûts de communication.

Exemple dans le domaine agricole : concevoir un système qui détecte des anomalies dans les mesures de capteurs agricoles, comme des variations inhabituelles d'humidité du sol ou de température. Plusieurs nœuds participent à l'entraînement du modèle, et les étudiants étudient les performances ainsi que les coûts de communication.

Projet 11 — Comparaison entre apprentissage centralisé et apprentissage fédéré

Implémenter une tâche de classification avec une approche centralisée et une approche fédérée. Comparer les performances prédictives, le coût de communication, le temps d'entraînement et l'impact d'une distribution non uniforme des données entre les clients.

Exemple dans le domaine agricole : comparer l'apprentissage centralisé et l'apprentissage fédéré pour une tâche de classification d'images de plantes saines ou malades, ou pour la prédiction des besoins en irrigation. Les données peuvent être réparties artificiellement entre plusieurs clients afin d'étudier l'effet de cette répartition sur les performances.

Les projets d'apprentissage fédéré pourront s'appuyer sur Flower et PyTorch. Les données pourront être publiques ou simulées, sous réserve de validation de l'enseignant. Une simulation sur une seule machine est acceptable si les clients et le serveur sont exécutés comme des processus ou des conteneurs indépendants.

4. Technologies et outils recommandés

Les étudiants sélectionneront les outils adaptés au sujet. L'objectif n'est pas d'utiliser toutes les technologies, mais de justifier les choix retenus.

Domaine	T**echnologies possibles**
Développement des services	Python, FastAPI, Flask, Java, Spring Boot
Communication synchrone	API REST/HTTP, gRPC
Communication asynchrone	RabbitMQ, Apache Kafka
Interfaces utilisateur	HTML, CSS, JavaScript, React ou équivalent
Conteneurisation	Docker
Déploiement multi-services	Docker Compose ; Kubernetes en extension
Stockage des données	PostgreSQL, MongoDB, Redis
LLM et RAG	API de LLM, Ollama, Hugging Face, bibliothèques de recherche documentaire
Apprentissage distribué	PyTorch, Flower
Tests et supervision	pytest, outils de test de charge, Prometheus et Grafana
Gestion de versions	Git et GitHub

Technologies exclues : les implémentations classiques fondées sur RPC traditionnel, Java RMI et CORBA ne constituent pas des solutions acceptées pour ce projet. Les étudiants devront privilégier les mécanismes modernes de communication, notamment REST, gRPC et la messagerie asynchrone.

5. Exigences techniques minimales

Quel que soit le sujet retenu, le projet devra respecter les exigences suivantes :

1. **Architecture distribuée réelle :** le système doit comporter plusieurs composants ou services indépendants qui communiquent à travers des interfaces définies. Une application monolithique simplement divisée en modules ne sera pas suffisante.
2. **Communication interservices :** au moins un mécanisme moderne de communication devra être implémenté et justifié.
3. **Interface utilisateur ou interface de démonstration :** une interface web, une API documentée ou une interface adaptée au sujet devra permettre de tester le système.
4. **Déploiement reproductible :** les composants devront pouvoir être lancés à partir d'instructions documentées, de préférence avec Docker Compose.
5. **Gestion des erreurs :** le système devra traiter au moins un scénario de panne ou d'indisponibilité d'un service.
6. **Tests :** des tests fonctionnels et des tests d'intégration devront être fournis. Une mesure de performance ou de charge devra être réalisée.
7. **Démonstration :** l'équipe devra présenter le fonctionnement du système et expliquer les choix d'architecture.

Pour les projets intégrant un LLM, les étudiants devront identifier le rôle exact du modèle, les entrées et sorties du service, ainsi que les limites de la solution. L'utilisation d'un LLM ne remplace pas la nécessité de démontrer une véritable architecture distribuée.

Pour les projets d'apprentissage distribué, l'équipe devra présenter la répartition des données et des calculs, le mécanisme d'agrégation et une évaluation expérimentale.

6. Étapes de réalisation

Étape 1 — Constitution des groupes et choix du sujet

Chaque équipe doit :

- constituer un groupe de 3 à 4 étudiants appartenant au même groupe de TD/TP ;
- désigner un responsable ;
- choisir un sujet parmi les propositions ou soumettre une proposition équivalente ;
- définir les fonctionnalités principales et les utilisateurs concernés ;
- préciser pourquoi une architecture distribuée est pertinente ;
- définir les technologies envisagées et les rôles des membres ;
- faire valider le sujet par l'enseignant.

Étape 2 — Conception de l'architecture

L'équipe doit produire :

- une description des composants et de leurs responsabilités ;
- un diagramme d'architecture présentant les services, les bases de données et les communications ;
- un diagramme de déploiement ;
- un diagramme de séquence illustrant un scénario important ;
- une description des principaux risques de panne et des solutions envisagées.

Étape 3 — Développement et intégration

L'équipe réalise les services, les mécanismes de communication, l'interface et les fonctionnalités spécifiques au sujet. Les développements doivent être intégrés progressivement dans le dépôt GitHub.

Étape 4 — Déploiement et expérimentation

L'équipe doit :

- déployer les différents composants ;
- vérifier leur communication ;
- exécuter des tests fonctionnels et d'intégration ;
- simuler au moins une panne ou une situation dégradée ;
- mesurer les performances et analyser les résultats obtenus.

Les mesures doivent être présentées sous forme de tableaux ou de graphiques, accompagnés d'une interprétation.

Étape 5 — Finalisation et soutenance

Chaque équipe prépare son rapport, sa présentation et une démonstration fonctionnelle. Les étudiants devront être capables d'expliquer l'architecture globale, les choix techniques, les résultats expérimentaux et les contributions individuelles.

7. Rapport du projet

Le rapport devra comporter les sections suivantes :

1. **Introduction et problématique :** contexte, objectifs et justification du recours à un système distribué.
2. **État de l'art et technologies :** présentation des solutions utilisées et justification des choix.
3. **Architecture logicielle :** description des composants, protocoles, échanges et diagrammes UML pertinents.
4. **Implémentation :** description des services, des interfaces, du stockage et, le cas échéant, des composants LLM ou d'apprentissage distribué.

5. **Déploiement :** configuration, conteneurs, réseau et instructions d'exécution.
6. **Tests et évaluation :** tests fonctionnels, scénarios de panne, mesures de performance et analyse des résultats.
7. **Limites et perspectives :** difficultés rencontrées, limites techniques et améliorations possibles.
8. **Conclusion et références :** synthèse des résultats et bibliographie.

Le rapport devra également présenter la répartition du travail et les contributions de chaque membre de l'équipe.

Longueur maximale : 20 pages.

8. Livrables obligatoires et modalités de remise

Chaque équipe devra disposer d'un dépôt GitHub contenant :

- le code source complet et son historique de développement ;
- le rapport du projet au format PDF, limité à 20 pages maximum ;
- la présentation au format PDF ;
- un fichier README contenant les prérequis et les instructions d'installation, de configuration et d'exécution ;
- les fichiers nécessaires au déploiement, notamment les fichiers Docker et Docker Compose lorsque ces outils sont utilisés ;
- les scripts de test et les résultats expérimentaux ;
- les données, configurations ou instructions permettant de reproduire les expériences, dans la mesure où leur redistribution est autorisée.

Le responsable de chaque groupe devra soumettre **le lien du dépôt GitHub sur Classroom**. Il lui appartient de vérifier que le dépôt contient tous les livrables demandés et que ceux-ci sont accessibles.

Calendrier à respecter

- **01 novembre 2026 :** date limite pour constituer les groupes et désigner leur responsable.
- **06 novembre 2026 :** date limite pour choisir et déclarer le sujet du projet.
- **13 décembre 2026 :** date limite de remise du projet sur Classroom.

Aucun dépôt ne sera accepté après le 13 décembre 2026. Les équipes doivent anticiper la finalisation du code, du rapport et de la présentation.

9. Soutenance et démonstration

Chaque groupe dispose de **15 minutes au total** :

- **10 minutes de présentation** : problématique, objectifs, architecture distribuée, technologies retenues, implémentation et résultats expérimentaux ;

- **5 minutes de démonstration** : présentation du fonctionnement de l'application et, selon le sujet, illustration d'un scénario de panne, d'un traitement par LLM ou d'un apprentissage distribué.

Chaque étudiant devra maîtriser les principes généraux du projet et être capable de présenter la partie dont il est responsable.

10. Critères d'évaluation proposés

Critère	Pondération indicative
Conception et pertinence de l'architecture distribuée	20 %
Qualité de l'implémentation et intégration des composants	25 %
Tests, gestion des pannes et évaluation expérimentale	20 %
Rapport et documentation technique	15 %
Soutenance et démonstration	10 %
Contribution individuelle et maîtrise du projet	10 %
Total	**100 %**

La note pourra tenir compte à la fois du résultat collectif et de la maîtrise individuelle. Les pondérations définitives seront précisées par l'enseignant.

11. Remarques importantes

- Les équipes doivent comporter 3 ou 4 étudiants appartenant au même groupe de TD/TP.
- Chaque groupe doit désigner un responsable chargé de soumettre le lien du dépôt GitHub sur Classroom.
- Chaque sujet doit être validé avant le début du développement.
- Les technologies classiques RPC, RMI et CORBA ne sont pas acceptées comme mécanismes principaux de communication.
- Le projet doit démontrer les propriétés et les difficultés d'un système distribué, et non uniquement présenter une application fonctionnelle.
- Les exemples agricoles proposés dans les catégories A, B, C et D sont facultatifs. Les étudiants peuvent choisir un autre domaine.
- Pour les projets LLM, les étudiants peuvent utiliser un modèle accessible par API ou exécuté localement, en précisant les éventuelles contraintes de disponibilité, de coût et de ressources.

- Pour les projets d'apprentissage distribué, les données et les ressources de calcul doivent rester compatibles avec les moyens disponibles.
- Toute réutilisation de code, de modèle ou de bibliothèque externe doit être documentée et référencée.
- Le plagiat, l'absence de contribution individuelle ou l'impossibilité de démontrer le fonctionnement du projet pourront affecter l'évaluation.

L'objectif final est de réaliser un prototype fonctionnel, documenté et expérimentalement évalué, illustrant les principes des systèmes distribués modernes et, selon le sujet choisi, leur interaction avec les LLM ou l'apprentissage distribué.